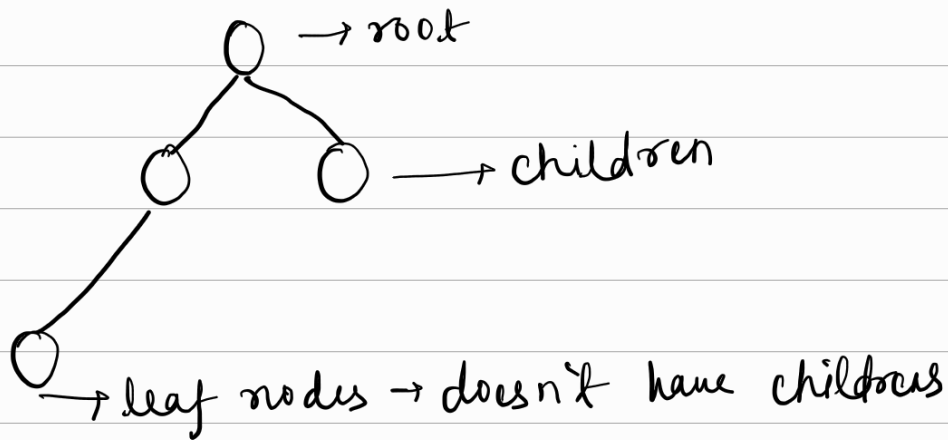


Striver Trees

→ Hierarchical data structures

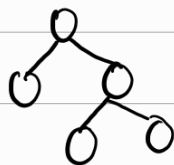


Subtree → specific section of tree →

Ancutors → all the parents of a node are called ancestors.

Types of Binary tree:

① Full binary tree: either have 0 or 2 children



② Complete Binary tree:

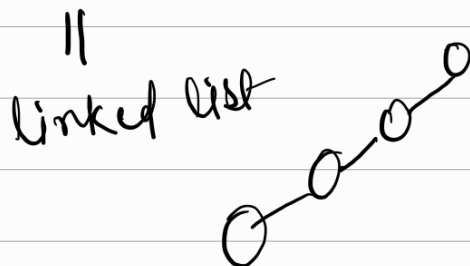
① All levels are completely filled except the last level.

② The last level has all nodes as left as possible.

③ Perfect Binary tree $\rightarrow$ all leaf nodes are at the same level.

④ Balanced Binary tree $\rightarrow$ height of tree at $\max \log(N)$
 $\downarrow$ nodes.

⑤ Degenerate tree $\rightarrow$ Skew tree



Binary tree representation in Python.

class Node:

```
def __init__(self, key):
```

```
    self.left = None
```

```
    self.right = None
```

```
    self.value = key
```

Traversal Technique (BFS/DFS)

DFS /

BFS

level order traversal

↓
per level print

↓
print the breadth.

Inorder Traversal

(left, root, right)

Pre order traversal

(root, left, right)

Post Order traversal

(left, right, root)

Code for all traversals

Preorder.

```
def preorder(key):  
    if key is None:  
        return  
    print(key.data, end = '')  
    preorder(key.left)  
    preorder(key.right)
```

PostOrder

```
def postorder(key):  
    if key is None:  
        return  
    postorder(key.left)  
    postorder(key.right)  
    print(key.data, end = '')
```

InOrder

```
def inorder(key):  
    if key is None:  
        return  
    inorder(key.left)  
    print(key.data, end = '')  
    inorder(key.right)
```

Level Order Traversal (BFS)

```
def bfs(root):  
    if root is None:  
        return  
    queue = [root]  
    while queue:  
        node = queue.pop(0)  
        print (node.data, end = " ")  
        if node.left:  
            queue.append (node.left)  
        if node.right:  
            queue.append (node.right)
```

Maximum Depth in Binary Tree | Height of Binary Tree

Traversals

Recursive / Level Order

$O(\text{height})$

space $O(n)$

def maxHeight(root):

→ height is number of edges in the tree,
from the root to the deepest node.

$1 + \max(l, r)$

def height(root):

if root is None:
return -1

lheight = height(root.left)

rheight = height(root.right)

return max(lheight, rheight) + 1

$O(n)$

check for → Balanced Binary Tree

Balanced BT → for every node,

$$\text{height}(\text{left}) - \text{height}(\text{right}) \leq 1$$

def check(node):

if node == None:

return 0

lh = check(node → left)

rh = check(node → right)

if lh == -1 or rh == -1 or abs(lh - rh) > 1:

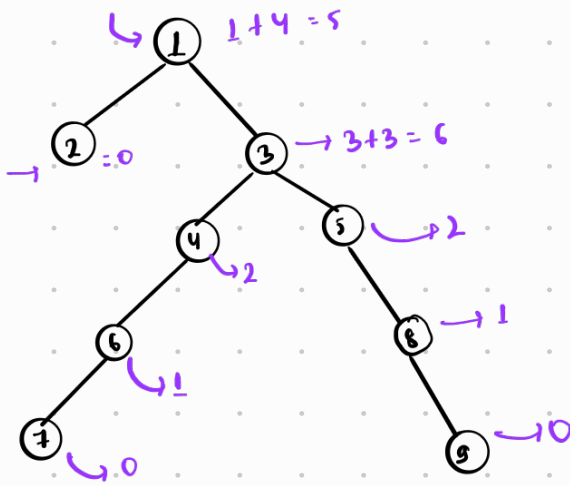
return -1

return max(lh, rh) + 1

return dfs(root) != -1

Diameter of Binary tree

- longest path between 2 nodes.
- path does not need to pass via root.



```
def findMax(node, maxi):
```

```
    if node == None:
```

```
        return
```

```
    lh = findMax(node.left,
```

```
                  maxi)
```

```
    rh = findMax(node.right,
```

```
                  maxi)
```

```
    maxi = max(maxi, lh+rh)
```

```
    return 1 + max(lh, rh)
```

Maximum Path Sum

node A to node B
↓
path



maximum height of binary tree
+ width of binary tree

maxi = float('-inf')

def dfs(node):

if node is None:

return 0

lg = max(dfs(node.left), 0)

rg = max(dfs(node.right), 0)

cp = node.val + rg + lg

self.maxi = max(self.maxi, cp)

return node.val + max(lg + rg)

dfs(root)

return self.maxi

18. Check if two trees are identical
or not

node val of tree1 == node val of tree2.

def areIdentical(tree1, tree2):

if tree1 is None or tree2 is None:

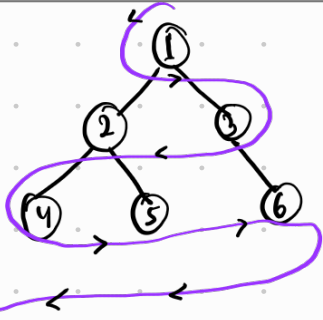
return tree1 == tree2

return tree1.val == tree2.val

and tree1.left == tree2.left

and tree1.right == tree2.right

19. Zig-zag or Spiral traversal.



1 → 3 → 2 → 4 → 5 → 6

we will use queue

11.4 code

from collections import deque

def zigzagLevelOrder(root):

if not root:

return []

queue = deque([root])

res = []

lto_r = True

while queue:

level = deque()

for i in range(len(queue)):

node = queue.popleft()

if lto_r:

level.append(node.val)

else:

level.appendleft(node.val)

if node.left:

queue.append(node.left)

if node.right:

queue.append(node.right)

result.append(list(level))

lto_r = not lto_r

return res

Faery ready.

from collections import deque

def zigzagLevelOrder(root):

if not root:

return []

queue = deque([root])

result = []

l-to-r = True

while queue:

level_size = len(queue)

level = []

for i in range(level_size):

node = queue.popleft()

level.append(node.val)

if node.left:

queue.append(node.left)

if node.right:

queue.append(node.right)

if not l-to-r:

level.reverse()

result.append(level)

l-to-r = not l-to-r

return result

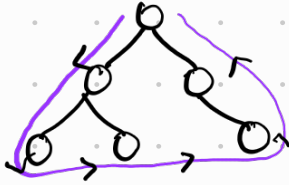
$T(n) = O(n)$
 $S(n) = O(n)$

$O(1)$ for collections.deque

not with
python list

Boundary Traversal in Binary Tree

The intuition is to do clockwise or anti-clockwise traversal of the binary tree. If we are doing anti-clockwise,



```
def boundaryOfBinaryTree(root):
```

```
    if not root:
        return []
```

```
    result = [root.val]
```

```
    def isleaf(node):
```

```
        return not node.left and not node.right
```

```
    def addLeftBoundary(node):
```

```
        while node:
```

```
            if not isleaf(node):
```

```
                result.append(node.val)
```

```
            node = node.left if node.left else node.right
```

```
    def addRightBoundary(node):
```

```
        temp = []
```

```
        while node:
```

```
            if not isleaf(node):
```

```
                temp.append(node.val)
```

```
            node = node.right if node.right else node.left
```

```
        result.extend(reversed(temp))
```

```
    def addLeaves(node):
```

```
        if not node: return
```

```
        if isleaf(node): result.append(node.val)
```

```
        else:
```

```
            addLeaves(node.left)
```

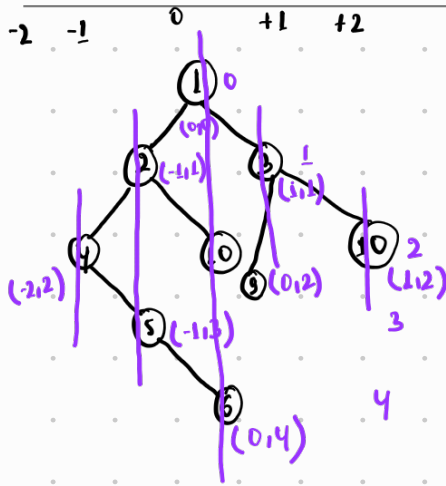
```
            addLeaves(node.right)
```

/ /

```
if root.left:
    addLeftBoundary(root.left)
addLums(root)
if root.right:
    addRightBoundary(root.right)

return result
```

Vertical Order Traversal



4
2 5
1 9 10 6
3
10

return list of lists

def verticalOrderTraversal(root):

nodes = []

def dfs(node, row, col):

if not node:

return

nodes.append((col, row, node.val))

dfs(node.left, row+1, col-1)

dfs(node.right, row+1, col+1)

dfs(root, 0, 0)

nodes.sort()

result = []

prev_col = None

for col, row, value in nodes:

if col != prev_col:

result.append([])

prev_col = col

result[-1].append(value)

return result

adding the
co-ordinates

moving left $\rightarrow -1$
right $\rightarrow +1$
down $\rightarrow +1$

storing column first, because
python sorts tuples lexicographically.

↓
nodes.sort()

↓
automatically sorts by

↓
column

↓
row

↓
value

tie breaker

top first, then
smaller value.

Flow

collect (col, row, value)

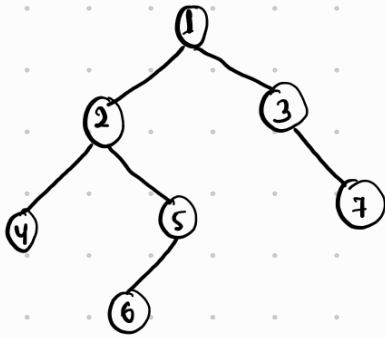
↓

sort

↓

group by col

Top View of Binary Tree



4 2 1 3 7

for top view, we do not store all nodes:

→ if a column is seen for the first time, store that node:

→ if the column already exists, skip it

```
from collections import deque
```

```
def topView(root):
```

```
    if not root:
```

```
        return []
```

```
    queue = deque([(root, 0)])  # (node, col)
```

```
    top_nodes = {}
```

```
    while queue:
```

```
        node, col = queue.popleft()
```

we are only keeping the first column encountered { if col not in top_nodes:

```
            top_nodes[col] = node.val
```

```
            if node.left:
```

```
                queue.append((node.left, col-1))
```

```
            if node.right:
```

```
                queue.append((node.right, col+1))
```

```
    result = []
```

```
    for col in sorted(top_nodes):
```

```
        result.append(top_nodes[col])
```

```
    return result
```

Mental model:

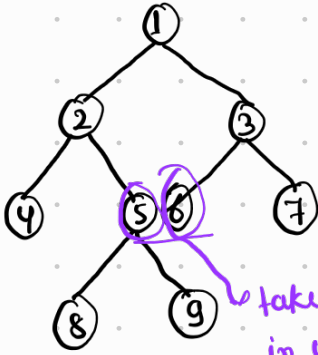
Vertical traversal → store every node in each column

Top View → store only the first node in each column

Bottom View → store the last node in each column

↓
next
↓

Bottom View of Binary Tree



BT $\rightarrow$ 4, 8, 6, 9, 7

because then every later node overwrites the earlier one, and the lowest visible node remains.

from collections import deque

def bottomView(root):

if not root:

return []

queue = deque([(root, 0)])

bottom_nodes = {}

while queue:

node, col = queue.popleft()

bottom_nodes[col] = node.val

if node.left:

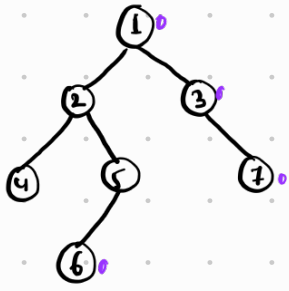
queue.append((node.left, col-1))

if node.right:

queue.append((node.right, col+1))

return [bottom_nodes[col] for col in sorted(bottom_nodes)]

Right / Left view of Binary tree



right view $\rightarrow 1 \ 3 \ 7 \ 6$

$\rightarrow$ last node of every level

```
def rightView(root):
```

```
    result = []
```

```
    def dfs(node, level):
```

```
        if not node:
```

```
            return
```

```
        if level == len(result):
```

```
            result.append(node.val)
```

```
        dfs(node.right, level+1)
```

```
        dfs(node.left, level+1)
```

```
    dfs(root, 0)
```

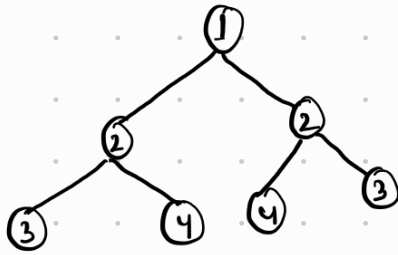
```
    return result
```

$TC = O(n)$

$SC = O(h)$

Check for Symmetrical Binary Tree

→ To check if it forms a mirror of itself around the center.



For every pair of node compared:

- values must be equal
- left's left == right's right
- left's right == right's left

```
def isSymmetric(root):
```

```
    if root is None:
```

```
        return
```

```
    def mirror(left, right):
```

```
        if not left and not right:
```

```
            return True
```

```
        if not left or not right:
```

```
            return False
```

```
        if left.val != right.val:
```

```
            return False
```

```
        return mirror(left.left, right.right)
```

```
            and
```

```
            mirror(left.right, right.left)
```

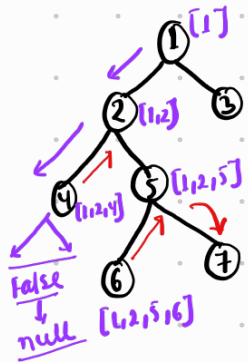
```
    return mirror(root.left, root.right)
```

$$T(c) = O(N)$$

$$S(c) = O(H)$$

Root to Node Path

→ node or node value will be given, you need to give the path to the node.



node = 7

Inorder traversal

if we encounter leaf, or null, we will return False, come back and go to right and check if we found the node.

[1,2,5,7] → found node → True

def rootToNodePath(root, target):

path = []

def dfs(node):

if not node:

return False

choose

path.append(node.val)

target found

if node.val == target:

return True

explore

if dfs(node.left) or dfs(node.right):

return True

target not found in this branch

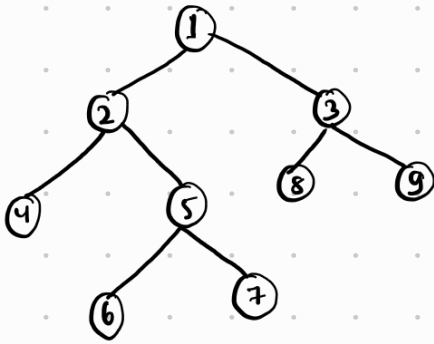
path.pop()

return False

dfs(root)

return path

27. Lowest Common Ancestor in Binary Tree



$$\text{lca}(4, 7) = 2$$

$$\text{lca}(5, 9) = 1$$

$$\text{lca}(2, 6) = 2$$

$\text{lca}(4, 7) = 2 \rightarrow \text{lca}$
 node 4 $(1, 2, 4)$
 node 7 $(1, 2, 5, 7)$

last element that matched in both node path will be lowest common ancestor

def lca (root, p, q):

if not root:
return None

if root == p or root == q:
return root

left = lca (root.left, p, q)

right = lca (root.right, p, q)

if left and right:
return root

return left if left else right } return left or right

if not root or root == p or root == q:
return root

if both targets eventually report back through diff branches, we have found their meeting point.

relationship

root to node path



which branch contains my target?

LCA

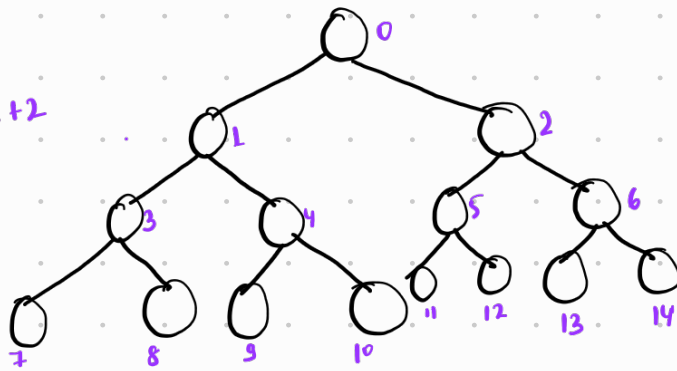


which branches contains my two target?

28. Maximum Width of Binary Tree

width $\rightarrow$ no of nodes in a level between any two nodes.

0 based indexing
 $2 \times i + 1$
segment trees



1 based indexing
 $2 \times i$
 $2 \times i + 1$

$\rightarrow$ give every node an index like it would have in a complete binary tree.
so if a node has index i ,
left child = $2 \times i + 1$ right child = $2 \times i + 2$

then for each level, width = last index - first index + 1

But, the indices can become very large for a deep tree, so at each level, normalize them by subtracting the first index.

so we use $\rightarrow$ current index = index - first index

from collections import deque

def widthOfBinaryTree(root):

if not root:

return 0

queue = deque([root, 0])

max_width = 0

while queue:

level_size = len(queue)

min_index = queue[0][1]

first = 0

last = 0

for i in range(level_size):

node, index = queue.popleft()

index -= min_index

if i == 0:

first = index

```
if i == level_size - 1:  
    last = index
```

/ /

```
if node.left:  
    queue.append((node.left, 2*index+1))  
  
if node.right:  
    queue.append((node.right, 2*index+2))  
  
max_width = max(max_width, last - first + 1)  
  
return max_width
```

29. Children Sum Property in Binary Tree

[node] = left child value + right child value

→ to make this possible, we can increment the value of node +1 n no of times.

There are two versions,

(1) Check for children sum property

```
def childrenSum(root):  
    if not root:  
        return False  
  
    if not root.left and not root.right:  
        return True  
  
    left_val = root.left.val if root.left else 0  
    right_val = root.right.val if root.right else 0  
    if root.val != left_val + right_val:  
        return False  
    return childrenSum(root.left) and  
           childrenSum(root.right)
```

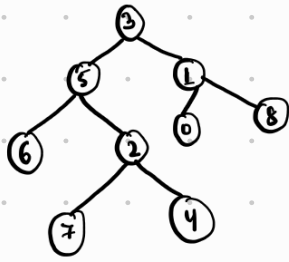
leaf node {
value extraction {
check children sum property {
recursive calls {

(2) Make/change the tree to make sure it follows children sum

```
def changeTree(root):  
    if not root: return  
    child-sum = 0  
    if root.left:  
        child-sum += root.left.val  
    if root.right:  
        child-sum += root.right.val  
    if child-sum >= root.val:  
        root.val = child-sum  
    else:  
        if root.left:  
            root.left.val = root.val  
        if root.right:  
            root.right.val = root.val
```

```
changeTree(root.left)  
changeTree(root.right)  
total = 0  
if root.left:  
    total += root.left.val  
if root.right:  
    total += root.right.val  
if root.left or root.right:  
    root.val = total
```

30. Print all the nodes at a distance of K in binary tree



k=2 target=5

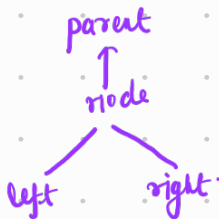
idea → tree normally let us move down to children,
but for "nodes at distance K", we may also need
to move up to the parent.

→ so we temporarily treat tree as undirected graph.

step 1 → create parent links

```
from collections import deque
def build_parent_map(root):
    parent = {}
    queue = deque([root])
    while queue:
        node = queue.popleft()
        if node.left:
            parent[node.left] = node
            queue.append(node.left)
        if node.right:
            parent[node.right] = node
            queue.append(node.right)
    return parent
```

now every node has three possible neighbors:

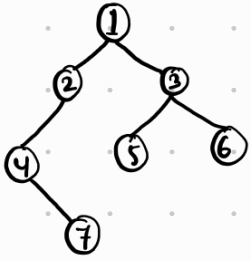


step 2 → BFS from the target

```
def distanceK(root, target, k):
    parent = build_parent_map(root)
    queue = deque([target])
    visited = {target}
    distance = 0
    while queue and distance < k:
        level_size = len(queue)
        for _ in range(level_size):
            node = queue.popleft()
            if node.left and node.left not in visited:
                visited.add(node.left)
                queue.append(node.left)
            if node.right and node.right not in visited:
                visited.add(node.right)
                queue.append(node.right)
            if node in parent and parent[node] not in visited:
                visited.add(parent[node])
                queue.append(parent[node])
        distance += 1
    return [node.val for node in queue]
```

distance + 1
return [node.val for node in queue]

31. Minimum time taken to burn the binary tree from a node.



→ basically nodes at distance K, without stopping at K.

→ same logic as last question, make the parent map.

```
now def burnTree(root, target):
    parent = build-parent-map(root)
    queue = deque([target])
    visited = {target}
    time = 0
    while queue:
        ls = len(queue)
        bnn = False
        for _ in range(ls):
            node = queue.popleft()
            left {
                if node.left and node.left not in visited:
                    visited.add(node.left)
                    queue.append(node.left)
                    bnn = True
            }
            right {
                if node.right and node.right not in visited:
                    visited.add(node.right)
                    queue.append(node.right)
                    bnn = True
            }
            parent {
                if node in parent and parent[node] not in visited:
                    visited.add(parent[node])
                    queue.append(parent[node])
                    bnn = True
            }
        if bnn:
            time += 1
    return time
```

32. Count total nodes in a complete Binary Tree

if the height of the leftmost path equals the height of the rightmost path, that subtree is a perfect binary tree.

and a perfect binary tree with height h has:

$$\{ 2^h - 1 \text{ nodes} \} \quad (2 \text{ to the power } h - 1 \text{ nodes})$$

```
def countNodes(root):  
    if not root:  
        return 0  
    lh = getLeftHeight(root)  
    rh = getRightHeight(root)  
    if lh == rh:  
        return (1 << lh) - 1    or    return 2**lh - 1  
    return 1 + countNodes(root.left) + countNodes(root.right)
```

```
def getLeftHeight(node):  
    height = 0  
    while node:  
        height += 1  
        node = node.left  
    return height
```

```
def getRightHeight(node):  
    height = 0  
    while node:  
        height += 1  
        node = node.right  
    return height
```

Construct a Binary Tree from Preorder and Inorder Traversal

preorder = [3, 9, 20, 15, 7]

inorder = [9, 3, 15, 20, 7]



preorder = root, left, right

inorder = left, root, right

→ preorder will give the root, inorder tells how to split left and right.

```
def buildTree(preorder, inorder):
```

```
    inorder_index = {
```

```
        value: i
```

```
        for i, value in enumerate(inorder)
```

```
    }
```

```
    preorder_index = 0
```

```
    def build(left, right):
```

```
        nonlocal preorder_index
```

```
        if left > right:
```

```
            return None
```

preorder → root

```
        root_value = preorder[preorder_index]
```

```
        preorder_index += 1
```

```
        root = TreeNode(root_value)
```

inorder → split point

```
        mid = inorder_index[root_value]
```

```
        root.left = build(left, mid-1)
```

```
        root.right = build(mid+1, right)
```

```
        return root
```

```
    return build(0, len(inorder)-1)
```

Construct the Binary Tree from Postorder and Inorder Traversal

Inorder = left, root, right

Postorder = left, right, root

postorder can give root, inorder can give split point

```
def build_tree(inorder, postorder):
```

```
    inorder_index = {
```

```
        value: i
```

```
        for i, value in enumerate(inorder)
```

```
    }
```

```
    post_index = len(postorder) - 1
```

```
    def build(left, right):
```

```
        if left > right:
```

```
            return None
```

```
        root_value = postorder[post_index]
```

```
        post_index -= 1
```

```
        root = TreeNode(root_value)
```

```
        mid = inorder_index[root_value]
```

```
        root.right = build(mid + 1, right)
```

```
        root.left = build(left, mid - 1)
```

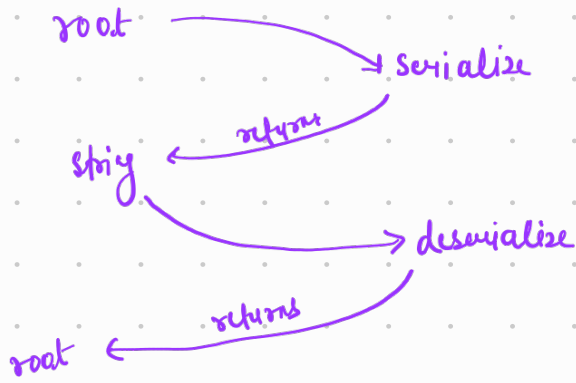
```
        return root
```

```
    return build(0, len(inorder) - 1)
```

imp. build right
first
because we consume
postorder from the
end.

Serialize and deserialize Binary Tree

/ /



from collections import deque

```
def serialize(root):  
    if not root:  
        return ""  
    queue = deque([root])  
    while queue:  
        node = queue.popleft()  
        if node is None:  
            result.append("#")  
        result.append(str(node.val))  
        queue.append(node.left)  
        queue.append(node.right)  
    return "".join(result)
```

```
def deserialize(data):  
    if not data:  
        return None  
    values = data.split(",")  
    root = TreeNode(int(values[0]))  
    queue = deque([root])  
    i = 1
```

while queue:

node = queue.popleft()

/ /

if values[i] != "#":

node.left = TreeNode(int(values[i]))

queue.append(node.left)

i += 1

if values[i] != "#":

node.right = TreeNode(int(values[i]))

queue.append(node.right)

i += 1

return root

Morris Traversal

(revisit)

→ Threaded Binary Tree

Morris Inorder

At current:

No left child?

Visit current
Move right

Has left child?

Find rightmost node in the
left subtree

No thread?

Create thread back to current
Move left

Thread already exist?

remove thread
visit current
move right

```
def morris_inorder(root):
```

```
    result = []
```

```
    current = root
```

```
    while current:
```

no left subtree { if current.left is None:

result.append(current.val)

current = current.right

else:

find inorder predecessor {

pred = current.left

while (pred.right and

pred.right != current):

pred = pred.right

first visit {

if pred.right is None:

pred.right = current

current = current.left

second visit {

else:

pred.right = None

result.append(current.val)

current = current.right

return result

Morris uses the rightmost node of the left subtree to create a temporary return path back to the current node, replacing the recursion stack.

Flatten a Binary Tree to Linked list.

Intuition: Morris Style

→ Rearrange the tree in place so it becomes a right only linked list following preorder traversal.

```
def flatten(root):  
    current = root  
    while current:  
        if current.left:  
            pred = current.left  
            while pred.right:  
                pred = pred.right  
            pred.right = current.right  
            current.right = current.left  
            current.left = None  
        current = current.right
```

Binary Tree from String done



muehehehe

1st Sept 2026